Contents

1 Basis

1.1 Default Code

```
// g++args: -std=c++17 -Wall -Wshadow -fsanitize=undefined
#pragma GCC optimize("O3,unroll-loops")
#pragma target optimize("avx2,bmi,bmi2,lzcnt,popcnt")
```

1.2 Some Useful Primes

```
/* 13331, 75577, 999983, 1097774749, 1076767633,
 * 100102021, 999997771, 1001010013, 1000512343,
 * 987654361, 999991231, 999888733, 98789101, 987777733,
 * 999991921, 1010101333, 1010102101, 1000000000039,
 * 1000000000000037, 2305843009213693951,
 * 4611686018427387847, 9223372036854775783,
 * 18446744073709551557 */
```

1.3 偽隨機 & MT19937

```
// chrono::steady_clock::now().time_since_epoch().count();
mt19937 gen(/*seed*/ hash<string>()("ShanC_orz"));
mt19937_64 mt(chrono::steady_clock::now().time_since_epoch
    ().count());
int randint(int a, int b) {  // [a, b]
  return uniform_int_distribution<int>(a, b)(gen);
}
// srand(time(NULL)), rand() % (b - a + 1) + a;
```

1.4 快讀快寫

```
inline int read() {
  char c = getchar();
  int x = 0, f = 1;
  while (c < '0' || c > '9') {
    if (c == '-') f = -1;
    c = getchar();
  }
  while (c >= '0' && c <= '9')
    x = x * 10 + c - '0', c = getchar();
  return x * f;
}
inline void write(int x) {
  if (x < 0) putchar('-'), x = -x;
  if (x > 9) write(x / 10);
  putchar(x % 10 + '0');
}
```

1.5 怪怪的東西

```
#define SECs ((double)clock() / CLOCKS_PER_SEC)
struct KeyHasher {
  size_t operator()(const Key& k) const {
    return k.first + k.second * 100000;
  }
};
typedef unordered_map<Key, int, KeyHasher> map_t;
__builtin_popcountll;  // 二進位有幾個1
__builtin_clzll;       // 左起第一個1之前0的個數
__builtin_parityll;    // popcount的奇偶性
__builtin_mul_overflow(a, b, &h)  // a*b是否溢位
```

2 Data Structures

2.1 Disjoint Set (併查集)

```
struct DSU {
  int n;
  vector<int> f, sz;
  DSU(int _n) : n(_n) {  // 初始化
    f.resize(n);
    sz.resize(n);
    for (int i = 0; i < n; i++) f[i] = i, sz[i] = 1;
  }
  int find(int x) {  // 返回 x 的根節點
    if (x == f[x]) return x;
    return f[x] = find(f[x]);
  }
  void merge(int x, int y) {  // 將 x 和 y 合併
    x = find(x), y = find(y);
    if (x == y) return;
    if (sz[x] < sz[y]) swap(x, y);  // 將小的併入大的
    sz[x] += sz[y];
    f[y] = x;
  }
};
```

2.2 Binary Indexed Tree

```
struct BIT {
  int n;
  vector<ll> bit;
  int lowbit(int x) { return x & -x; }
  BIT(int _n) : n(_n + 1) { bit = vector<ll>(_n + 1, 0); }
  void update(int x, ll v) {  // 將 x 的值加上 v
    for (; x < n; x += lowbit(x)) bit[x] += v;
  }
  ll query(int x) {  // 查詢區間 [1, x] 的總和
    ll ret = 0;
    for (; x; x -= lowbit(x)) ret += bit[x];
    return ret;
  }
  ll query(int l, int r) {  // 查詢區間 [l, r] 的總和
    return query(r) - query(l - 1);
  }
  int kth(int k) {
    int x = 0;
    for (int i = 1 << __lg(n); i; i >>= 1)
      if (x + i <= n && k >= bit[x + i - 1])
        x += i, k -= bit[x - 1];
    return x;
  }
};
```

2.3 二維 BIT

```
struct BIT {
  int n;
  vector<vector<ll>> bit;
```

```
  int lowbit(int x) { return x & -x; }
  BIT(int _n) : n(_n + 1) {
    bit.assign(n, vector<ll>(n, 0));
  }
  void update(int x, int y, ll val) {  // 更新 (x, y)
    for (int i = x; i < n; i += lowbit(i))
      for (int j = y; j < n; j += lowbit(j))
        bit[i][j] += val;
  }
  ll query(int x, int y) {  // 查詢 (1, 1) ~ (x, y) 的和
    ll ans = 0;
    for (int i = x; i; i -= lowbit(i))
      for (int j = y; j; j -= lowbit(j)) ans += bit[i][j];
    return ans;
  }
  ll range_query(int a, int b, int x, int y) {
    return query(x, y) - query(x, b - 1) -
           query(a - 1, y) + query(a - 1, b - 1);
  }
};
```

2.4 Segment Tree

```
#define cl (i << 1)
#define cr (i << 1 | 1)
#define NO_TAG 0
struct SegmentTree {  // 1-base
  int n;
  vector<int> seg, tag;
  SegmentTree(int _n) : n(_n) {  // 空的 SegmentTree
    seg.resize(n * 4), tag.resize(n * 4);
  }
  void push(int i, int l, int r) {
    if (tag[i] != NO_TAG) {
      seg[i] += tag[i];  // update by tag
      if (l != r)
        tag[cl] += tag[i], tag[cr] += tag[i];  // push
      tag[i] = 0;
    }
  }
  void pull(int i, int l, int r) {
    int mid = (l + r) >> 1;
    push(cl, l, mid), push(cr, mid + 1, r);
    seg[i] = max(seg[cl], seg[cr]);  // pull (操作改這)
  }
  void build(int i, int l, int r) {
    if (l == r) {
      seg[i] = arr[l];  // 初始值
      return;
    }
    int mid = (l + r) >> 1;
    build(cl, l, mid), build(cr, mid + 1, r);
    pull(i, l, r);
  }
  void update(int i, int l, int r, int ql, int qr,
              int v) {
    push(i, l, r);
    if (ql <= l && r <= qr) {
      tag[i] += v;
      return;
    }
    int mid = (l + r) >> 1;
    if (ql <= mid) update(cl, l, mid, ql, qr, v);
    if (qr > mid) update(cr, mid + 1, r, ql, qr, v);
    pull(i, l, r);
  }
  int query(int i, int l, int r, int ql, int qr) {
    push(i, l, r);
    if (ql <= l && r <= qr) return seg[i];
    int mid = (l + r) >> 1, ans = -INF;
    if (ql <= mid)  // (操作改這)
      ans = max(ans, query(cl, l, mid, ql, qr));
    if (qr > mid)  // (操作改這)
      ans = max(ans, query(cr, mid + 1, r, ql, qr));
    return ans;
  }
  // 區間 [ql, qr] 加值 v
  void update(int ql, int qr, int v) {
    update(1, 1, n, ql, qr, v);
  }
  // 查詢區間 [ql, qr]
  int query(int ql, int qr) {
    return query(1, 1, n, ql, qr);
  }
};
```

2.5 Treap

```
struct Treap {
  int sz, val, pri, tag;
  Treap *l, *r;
  Treap(int _val) : sz(1), val(_val), tag(0) {
    l = r = NULL, pri = rand();  // 隨機產生優先度
  }
};
int size(Treap* a) { return a ? a->sz : 0; }
void pull(Treap* a) {
  a->sz = size(a->l) + size(a->r) + 1;
}
Treap* merge(Treap* a, Treap* b) {
  // val of a is always bigger than val of b
  if (!a || !b) return a ? a : b;
  if (a->pri > b->pri) {
    a->r = merge(a->r, b);
    pull(a);
    return a;
  } else {
    b->l = merge(a, b->l);
    pull(b);
    return b;
  }
}
void split(Treap* t, int k, Treap*& a, Treap*& b) {
  // a<k, b>=k
  if (!t) {
    a = b = NULL;
    return;
  }
  if (k <= t->val) {
    b = t;
    split(t->l, k, a, b->l);
    pull(b);
  } else {
    a = t;
    split(t->r, k, a->r, b);
    pull(a);
  }
}
Treap* add(Treap* t, int v) {
  Treap *val = new Treap(v), *l = NULL, *r = NULL;
  split(t, v, l, r);
  return merge(merge(l, val), r);
}
Treap* del(Treap* t, int v) {
  Treap *l, *mid, *r, *temp;
  split(t, v, l, temp);
  split(temp, v + 1, mid, r);
  return merge(l, r);
}
int position(Treap* t, int p) {  // base 1
  if (size(t->l) + 1 == p) return t->val;
  if (size(t->l) < p)
    return position(t->r, p - size(t->l) - 1);
  return position(t->l, p);
}
int query(Treap* t, int k) {  // num of >= k
  if (!t) return 0;
  if (t->val == k) return size(t->l) + 1;
  if (t->val > k) return query(t->l, k);
  return size(t->l) + 1 + query(t->r, k);
}
```

2.6 平板電腦

```
#include <bits/extc++.h>

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/rope>
#include <functional>
#include <queue>
using namespace std;
using namespace __gnu_cxx;
using namespace __gnu_pbds;
typedef tree<int, null_type, less<int>, rb_tree_tag,
             tree_order_statistics_node_update>
    set_t;
typedef cc_hash_table<int, int> umap_t;
```

```
typedef priority_queue<int> heap;

set_t s;  // Insert some entries into s.
s.insert(12), s.insert(505);
// The order of the keys should be: 12, 505.
assert(*s.find_by_order(0) == 12);
assert(*s.find_by_order(3) == 505);
// The order of the keys should be: 12, 505.
assert(s.order_of_key(12) == 0);
assert(s.order_of_key(505) == 1);
s.erase(12);  // Erase an entry.
// The order of the keys should be: 505.
assert(*s.find_by_order(0) == 505);
// The order of the keys should be: 505.
assert(s.order_of_key(505) == 0);
heap h1, h2;
h1.join(h2);
rope<char> r[2];
r[1] = r[0];  // persistenet
string t = "abc";
r[1].insert(0, t.c_str());
r[1].erase(1, 1);
cout << r[1].substr(0, 2);
```

3 Mathematics

3.1 公式們

$$\sum_{k=1}^{n} n = \frac{n(n+1)}{2}, \quad \sum_{k=1}^{n} n^2 = \frac{n(n+1)(2n+1)}{6}, \quad \sum_{k=1}^{n} n^3 = \left[\frac{n(n+1)}{2}\right]^2$$

$$\sum_{k=1}^{n} n^4 = n(1+n)(1+2n)(-1+3n+3n^2)/30$$

$$\sum_{k=1}^{n} n^5 = n^2(n+1)^2(2n^2+2n-1)/12$$

錯位排列 $$DP_i = \begin{cases} 0 \vee 1, & i = 1 \vee 2 \\ (i-1)(DP_{i-1} \times DP_{i-2}), & i \geq 3 \\ = iDP_{i-1} + (-1)^i & \text{另一種關係式} \end{cases}$$

生成樹數量 n^{n-2}

$$P_k^n = \frac{n!}{k!(n-k)!}, \quad C_k^n = \frac{n!}{(n-k)!}, \quad H_k^n = C_{n-1}^{n+k-1}$$

$$\pi = \arccos(-1), \quad e^{i\theta} = \cos\theta + i\sin\theta, \quad pV = nTR$$

$$a^{\varphi(m)} \equiv 1 (\bmod m), \quad a^b \equiv a^{b \bmod \varphi(m)-1} (\bmod m)$$

$$\varphi(m) = m-1, \text{When } m \text{ is Prime}$$

3.2 階乘 & 模逆元

```
ll fac[MXN], inv[MXN];
void init() {
  fac[0] = 1;  // 0! = 1
  for (ll i = 1; i < MXN; i++)
    fac[i] = fac[i - 1] * i % MOD;  // n! = n * (n-1)!
  inv[MXN - 1] = power(fac[MXN - 1], MOD - 2);
  for (ll i = MXN - 2; i >= 0; i--)
    inv[i] = inv[i + 1] * (i + 1) % MOD;  // (n!)^(-1)
}
```

3.3 GCD & LCM

```
ll gcd(ll a, ll b) {
  if (b && a)  // 輾轉先除法 -> 交叉取餘數直到不能再取
    while ((a %= b) && (b %= a)) {}
  return a + b;
}
ll lcm(ll a, ll b) { return a * b / gcd(a, b); }
```

3.4 EXGCD

$ax + by = gcd(a, b)$

```
int exgcd(int a, int b, ll& x, ll& y) {
  if (b == 0) {
    x = 1, y = 0;
    return a;
  }
  int now = exgcd(b, a % b, y, x);
  y -= a / b * x;
  return now;
}
```

3.5 Fast Power

```
ll power(ll x, ll y, ll mod) {
  ll ret = 1;
  while (y) {
    if (y & 1) ret = ret * x % mod;
    x = x * x % mod, y >>= 1;
  }
  return ret;
}
```

3.6 矩陣乘法

```
struct Mat {  // Mat t(r, c);
  ll a[200][200], r, c;
  Mat(int _r, int _c) : r(_r), c(_c) {
    memset(a, 0, sizeof(a));
  }
  void build() {  // 單位矩陣
    for (int i = 0; i < r; ++i) a[i][i] = 1;
  }
};
Mat operator*(Mat x, Mat y) {
  Mat z(x.r, y.c);
  for (int i = 0; i < x.r; ++i)
    for (int j = 0; j < x.c; ++j)
      for (int k = 0; k < y.c; ++k)
        (z.a[i][j] += x.a[i][k] * y.a[k][j] % MOD) %= MOD;
  return z;
}
```

3.7 FFT

使用前要先跑過 pre_fft()；n 必須是 2^k。

```
const int MAXN = 262144 << 1;  // (must be 2^k)
// before any usage, run pre_fft() first
// typedef long double ld;
typedef complex<ld> cplx;  // real(), imag()
const ld PI = acosl(-1);
const cplx I(0, 1);
cplx omega[MAXN + 1];
void pre_fft() {
  for (int i = 0; i <= MAXN; i++)
    omega[i] = exp(i * 2 * PI / MAXN * I);
}
void fft(int n, cplx a[], bool inv = false) {
  int basic = MAXN / n;
  int theta = basic;
  for (int m = n; m >= 2; m >>= 1) {
    int mh = m >> 1;
    for (int i = 0; i < mh; i++) {
      cplx w = omega[inv ? MAXN - (i * theta % MAXN)
                         : i * theta % MAXN];
      for (int j = i; j < n; j += m) {
        int k = j + mh;
        cplx x = a[j] - a[k];
        a[j] += a[k];
        a[k] = w * x;
      }
    }
    theta = (theta * 2) % MAXN;
  }
  int i = 0;
  for (int j = 1; j < n - 1; j++) {
    for (int k = n >> 1; k > (i ^= k); k >>= 1) {}
    if (j < i) swap(a[i], a[j]);
  }
  if (inv)
    for (i = 0; i < n; i++) a[i] /= n;
}
cplx arr[MAXN + 1];
inline void mul(int _n, ll a[], int _m, ll b[],
                ll ans[]) {
  int n = 1, sum = _n + _m - 1;
  while (n < sum) n <<= 1;
  for (int i = 0; i < n; i++) {
    double x = (i < _n ? a[i] : 0),
           y = (i < _m ? b[i] : 0);
    arr[i] = complex<double>(x + y, x - y);
  }
  fft(n, arr);
  for (int i = 0; i < n; i++) arr[i] = arr[i] * arr[i];
  fft(n, arr, true);
  for (int i = 0; i < sum; i++)
```

```
    ans[i] = (long long int)(arr[i].real() / 4 + 0.5);
}
```

3.8 質數表

```
// notPrime, 最大質因數, 質因數數量
int notPrime[MXN], fac[MXN], num[MXN];
void getPrimes() {
  notPrime[1] = 1, fac[1] = num[1];
  for (int i = 2; i < MXN; ++i) {
    if (notPrime[i]) continue;
    for (int j = i; j < MXN; j += i) {
      if (i != j) notPrime[j] = 1;
      fac[j] = i, num[j]++;
    }
  }
}
vector<int> ret;  // 質因數分解
void div(int x) {
  for (; x > 1; x /= fac[x]) ret.push_back(fac[x]);
}
```

3.9 歐拉函數

$\varphi(x) =$ 所有小於等於 x 且和 x 互質的數的數量

```
int _phi(int n) {  // O(sqrtN)
  int res = n, a = n;
  for (int i = 2; i * i <= a; i++) {
    if (a % i == 0) {
      res = res / i * (i - 1);
      while (a % i == 0) a /= i;
    }
  }
  if (a > 1) res = res / a * (a - 1);
  return res;
}

int phi[MXN];  // 建表 最大 1e7
void phi_table(int n) {
  phi[1] = 1;
  for (int i = 2; i <= n; ++i) {
    if (phi[i]) continue;
    for (int j = i; j <= n; j += i) {
      if (phi[j] == 0) phi[j] = j;
      phi[j] = phi[j] / i * (i - 1);
    }
  }
}
```

3.10 Miller-Rabin Algorithm

$$Magic = \begin{cases} \{2,7,61\}, & n < 4,759,123,141 \\ \{2,13,23,1662803\}, & n < 1,122,004,669,633 \\ \{2,3,5,7,11,13\}, & n < 3,474,749,660,383 \\ \{2,325,9375,28178, & n < 2^{64} \\ 450775,9780504,1795265022\} & \end{cases}$$

```
ll mul(ll x, ll y, ll mod) {
  ll ret = x * y - (ll)((long double)x / mod * y) * mod;
  return ret < 0 ? ret + mod : ret;
  // return x * y % mod; // for __int128
}
ll magic[] = {2, 7, 61};  // 這邊填入要用於測試的數列
ll S = 3;                 // 測試的數列數字的數量
bool witness(ll a, ll n, ll u, int t) {
  if (!a) return 0;
  ll x = power(a, u, n);
  for (int i = 0; i < t; i++) {
    ll nx = mul(x, x, n);
    if (nx == 1 && x != 1 && x != n - 1) return 1;
    x = nx;
  }
  return x != 1;
}
bool miller_rabin(ll n) {
  int s = S;  // magic number size
  // iterate s times of witness on n
  if (n < 2) return false;
  if (!(n & 1)) return (n == 2);

  // 將 n - 1 寫成 u * (2 ^ t) 的形式
  ll u = n - 1;
  int t = 0;
  while (!(u & 1)) u >>= 1, t++;

  while (s--) {
    ll a = magic[s] % n;
    if (witness(a, n, u, t)) return false;
  }
  return true;
}
```

3.11 Pollard's Rho Algorithm

```
vector<ll> ret;
ll f(ll x, ll c, ll mod) {
  return (mul(x, x, mod) + c) % mod;
}
ll pollard_rho(ll n) {
  ll c = 1, x = 0, y = 0, p = 2, q, t = 0;
  while (t++ % 128 || gcd(p, n) == 1) {
    if (x == y) c++, y = f(x = 2, c, n);
    if (q = mul(p, abs(x - y), n)) p = q;
    x = f(x, c, n);
    y = f(f(y, c, n), c, n);
  }
  return gcd(p, n);
}
void fact(ll x) {  // 透過遞迴找質數
  if (miller_rabin(x)) {
    ret.push_back(x);
    return;
  }
  ll f = pollard_rho(x);
  fact(f);
  fact(x / f);
}
```

3.12 高斯消去法

```
/**
 * mat[n][m] 為 n 個 m 元一次連例方程式的增廣矩陣
 * ans[m] 輸出 x_m 的解
 * led[n] 紀錄第 n 列的領導係數的位置
 * 若無解則回傳 false
 */
ll mat[MXN][MXN], ans[MXN], led[MXN];
bool gaussian(int n, int m) {
  for (int i = 0; i < n; i++) {  // 高斯消元法
    // 第 i 列的領導項
    while (led[i] <= m && mat[i][led[i]] == 0) led[i]++;
    // 當每項係數皆為 0 時，mat_{im} == b_i != 0 => 無解
    if (led[i] == m && mat[i][m] != 0) return false;
    int c = led[i];
    for (int j = 0; j < n; j++) {
      // 消掉第 j 列的這項係數，使 mat_{jc} = 0
      if (i == j) continue;
      int r = mat[j][c] / mat[i][c];
      // mat_j -= mat_i * (mat_{jc} / mat_{ic})
      for (int k = c; k <= m; k++)
        mat[j][k] = mat[j][k] - mat[i][k] * r;
    }
  }
  for (int i = 0; i < m; i++) {
    // 紀錄 x_c 的解為 b_i / mat_{ic}
    ans[led[i]] = mat[i][m] / mat[i][led[i]];
  }
  return true;  // 有解
}
```

3.13 約瑟夫問題

```
int josephus(int n, int m) {  // n 人每 m 次
  int ans = 0;
  for (int i = 1; i <= n; ++i) ans = (ans + m) % i;
  return ans;
}
```

3.14 莫比烏斯函數

```
vector<int> pri;
bool not_prime[MXN];
int mu[MXN];

void pre(int n) {
```

```
  mu[1] = 1;
  for (int i = 2; i <= n; ++i) {
    if (!not_prime[i]) {
      mu[i] = -1;
      pri.push_back(i);
    }
    for (int pri_j : pri) {
      if (i * pri_j > n) break;
      not_prime[i * pri_j] = true;
      if (i % pri_j == 0) {
        mu[i * pri_j] = 0;
        break;
      }
      mu[i * pri_j] = -mu[i];
    }
  }
}
```

4 Graph

4.1 Topological Sort

```
vector<int> edge[MAXN], ans;
int deg[MAXN];      // in degree
void topo(int n) {  // 1-base
  queue<int> que;
  for (int i = 1; i <= n; i++)
    if (!deg[i]) que.push(i);
  while (!que.empty()) {
    int u = que.front();
    que.pop();
    ans.push_back(u);
    // 結果存 ans
    for (int v : edge[u]) {
      deg[v]--;
      if (!deg[v]) que.push(v);
    }
  }
}
```

4.2 Tarjan's Algorithm for BCC

```
struct BccVertex {
  int n, nScc, step, dfn[MXN], low[MXN];
  vector<int> E[MXN], sccv[MXN];
  int top, stk[MXN];
  void init(int _n) {
    n = _n;
    nScc = step = 0;
    for (int i = 0; i < n; i++)
      E[i].clear();
  }
  void addEdge(int u, int v) {
    E[u].PB(v);
    E[v].PB(u);
  }
  void DFS(int u, int f) {
    dfn[u] = low[u] = step++;
    stk[top++] = u;
    for (auto v : E[u]) {
      if (v == f) continue;
      if (dfn[v] == -1) {
        DFS(v, u);
        low[u] = min(low[u], low[v]);
        if (low[v] >= dfn[u]) {
          int z;
          sccv[nScc].clear();
          do {
            z = stk[--top];
            sccv[nScc].PB(z);
          } while (z != v);
          sccv[nScc++].PB(u);
        }
      } else low[u] = min(low[u], dfn[v]);
    }
  }
  vector<vector<int>> solve() {
    vector<vector<int>> res;
    for (int i = 0; i < n; i++)
      dfn[i] = low[i] = -1;
    for (int i = 0; i < n; i++)
      if (dfn[i] == -1) {
        top = 0;
        DFS(i, i);
      }
    REP(i, nScc) res.PB(sccv[i]);
    return res;
  }
} graph;
```

4.3 Bellmen-Ford Algorithm

```
struct Edge {
  int from, to, weight;
} edge[MAXN];
int dis[MAXN];
void bellmen_ford(int n, int m) {
  dis[1] = 0;  // 起點的距離一定是 0
  for (int j = 0; j < n - 1;
       j++) {  // n 個節點要跑 n - 1 次
    for (int i = 0; i < m; i++) {  // 對於所有邊都嘗試鬆弛
      if (dis[edge[i].to] >
          dis[edge[i].from] + edge[i].weight)
        dis[edge[i].to] =
            dis[edge[i].from] + edge[i].weight;
    }
  }
}
```

4.4 Dijkstra's Algorithm

```
vector<pii> vec[N];  // vec[u] = {v, w}: u 為起點 v 為終點
                     // w 為路權
bool vis[N];                    // 紀錄該節點是否走過
vector<int> dijkstra(int s) {  // 起點
  vector<int> dis(N);
  for (int i = 0; i < N; i++)  // 初始化
    dis[i] =
        INF;  // 值要設為比可能的最短路徑權重還要大的值
  dis[s] = 0;
  priority_queue<pii, vector<pii>, greater<pii>>
      pq;  // 以小到大排序
  // {w, v}: w 為路權 v 為節點
  pq.push({dis[s], s});
  while (pq.empty() == 0) {
    int u = pq.top().second;
    pq.pop();
    if (vis[u])  // 走過的不要再走
      continue;
    vis[u] = 1;
    for (auto i : vec[u]) {
      int v = i.first, w = i.second;
      if (dis[u] + w < dis[v]) {  // 鬆弛
        dis[v] = dis[u] + w;
        pq.push({dis[v], v});
      }
    }
  }
  return dis;
}
```

4.5 Dinic's Algorithm for Maximum Flow

```
#define SZ(x) ((int)x.size())
#define PB push_back
struct Dinic {
  struct Edge {
    int v, f, re;
  };
  int n, s, t, level[MXN];
  vector<Edge> E[MXN];
  void init(int _n, int _s, int _t) {
    n = _n;
    s = _s;
    t = _t;
    for (int i = 0; i < n; i++) E[i].clear();
  }
  void add_edge(int u, int v, int f) {
    E[u].PB({v, f, SZ(E[v])});
    E[v].PB({u, 0, SZ(E[u]) - 1});
  }
  bool BFS() {
    for (int i = 0; i < n; i++) level[i] = -1;
    queue<int> que;
    que.push(s);
    level[s] = 0;
    while (!que.empty()) {
```

```
      int u = que.front();
      que.pop();
      for (auto it : E[u]) {
        if (it.f > 0 && level[it.v] == -1) {
          level[it.v] = level[u] + 1;
          que.push(it.v);
        }
      }
    }
    return level[t] != -1;
  }
  int DFS(int u, int nf) {
    if (u == t) return nf;
    int res = 0;
    for (auto &it : E[u]) {
      if (it.f > 0 && level[it.v] == level[u] + 1) {
        int tf = DFS(it.v, min(nf, it.f));
        res += tf;
        nf -= tf;
        it.f -= tf;
        E[it.v][it.re].f += tf;
        if (nf == 0) return res;
      }
    }
    if (!res) level[u] = -1;
    return res;
  }
  int flow(int res = 0) {
    while (BFS()) res += DFS(s, 2147483647);
    return res;
  }
} flow;
/*
    查找二分圖配對點:
    在 add_edge 時寫上 index[u].push_back({v,
   flow.E[u].size()});

    在 flow 完之後寫上這個
    for (int i = 1; i <= n; i++) { // 遍歷所有節點
        for (pair<int, int> j : index[i]) {
            if (flow.E[i][j.second - 1].f == 0)
                cout << i << ' ' << j.first << '\n';
        }
    }
 */
```

4.6 Dominator Tree

```
#define REP(i, s, e) for (int i = (s); i <= (e); i++)
#define REPD(i, s, e) for (int i = (s); i >= (e); i--)
struct DominatorTree {  // O(N)
  int n, s;
  vector<int> g[MAXN], pred[MAXN];
  vector<int> cov[MAXN];
  int dfn[MAXN], nfd[MAXN], ts;
  int par[MAXN];  // idom[u] s到u的最後一個必經點
  int sdom[MAXN], idom[MAXN];
  int mom[MAXN], mn[MAXN];
  inline bool cmp(int u, int v) {
    return dfn[u] < dfn[v];
  }
  int eval(int u) {
    if (mom[u] == u) return u;
    int res = eval(mom[u]);
    if (cmp(sdom[mn[mom[u]]], sdom[mn[u]]))
      mn[u] = mn[mom[u]];
    return mom[u] = res;
  }
  void init(int _n, int _s) {
    ts = 0;
    n = _n;
    s = _s;
    REP(i, 1, n) g[i].clear(), pred[i].clear();
  }
  void addEdge(int u, int v) {
    g[u].push_back(v);
    pred[v].push_back(u);
  }
  void dfs(int u) {
    ts++;
    dfn[u] = ts;
    nfd[ts] = u;
    for (int v : g[u])
      if (dfn[v] == 0) {
        par[v] = u;
        dfs(v);
      }
  }
  void build() {
    REP(i, 1, n) {
      dfn[i] = nfd[i] = 0;
      cov[i].clear();
      mom[i] = mn[i] = sdom[i] = i;
    }
    dfs(s);
    REPD(i, n, 2) {
      int u = nfd[i];
      if (u == 0) continue;
      for (int v : pred[u])
        if (dfn[v]) {
          eval(v);
          if (cmp(sdom[mn[v]], sdom[u]))
            sdom[u] = sdom[mn[v]];
        }
      cov[sdom[u]].push_back(u);
      mom[u] = par[u];
      for (int w : cov[par[u]]) {
        eval(w);
        if (cmp(sdom[mn[w]], par[u])) idom[w] = mn[w];
        else idom[w] = par[u];
      }
      cov[par[u]].clear();
    }
    REP(i, 2, n) {
      int u = nfd[i];
      if (u == 0) continue;
      if (idom[u] != sdom[u]) idom[u] = idom[idom[u]];
    }
  }
} domT;
```

4.7 Edmonds Blossom Algorithm

```
namespace blossom {
#define d(x) (lab[x.u] + lab[x.v] - 2 * e[x.u][x.v].w)
#define all(x) x.begin(), x.end()
const ll N = 403 * 2;
const ll inf = 1e18;
struct Q {
  ll u, v, w;
} e[N][N];
vector<ll> p[N];
ll n, m = 0, id, h, t, lk[N], sl[N], st[N], f[N], b[N][N],
      s[N], ed[N], q[N], lab[N];
void upd(ll u, ll v) {
  if (!sl[v] || d(e[u][v]) < d(e[sl[v]][v])) sl[v] = u;
}
void ss(ll v) {
  sl[v] = 0;
  for (ll u = 1; u <= n; ++u)
    if (e[u][v].w > 0 && st[u] != v && !s[st[u]])
      upd(u, v);
}
void ins(ll u) {
  if (u <= n) q[++t] = u;
  else
    for (ll v : p[u]) ins(v);
}
void ch(ll u, ll w) {
  st[u] = w;
  if (u > n)
    for (ll v : p[u]) ch(v, w);
}
ll gr(ll u, ll v) {
  if ((v = find(all(p[u]), v) - p[u].begin()) & 1) {
    reverse(1 + all(p[u]));
    return (ll)p[u].size() - v;
  }
  return v;
}
void stm(ll u, ll v) {
  lk[u] = e[u][v].v;
  if (u <= n) return;
  Q w = e[u][v];
  ll x = b[u][w.u], y = gr(u, x);
  for (ll i = 0; i < y; ++i) stm(p[u][i], p[u][i ^ 1]);
  stm(x, v);
```

```
  rotate(p[u].begin(), y + all(p[u]));
}
void aug(ll u, ll v) {
  ll w = st[lk[u]];
  stm(u, v);
  if (!w) return;
  stm(w, st[f[w]]);
  aug(st[f[w]], w);
}
ll lca(ll u, ll v) {
  for (id++; u | v; swap(u, v)) {
    if (!u) continue;
    if (ed[u] == id) return u;
    ed[u] = id;
    if (u = st[lk[u]]) u = st[f[u]];  // =, not ==
  }
  return 0;
}
void add(ll u, ll a, ll v) {
  ll x = n + 1;
  while (x <= m && st[x]) ++x;
  if (x > m) ++m;
  lab[x] = s[x] = st[x] = 0;
  lk[x] = lk[a];
  p[x].clear();
  p[x].push_back(a);
#define op(q)                                          \
  for (ll i = q, j = 0; i != a; i = st[f[j]])          \
  p[x].push_back(i), p[x].push_back(j = st[lk[i]]), \
      ins(j)  // also not ==
  op(u);
  reverse(1 + all(p[x]));
  op(v);
  ch(x, x);
  for (ll i = 1; i <= m; ++i) e[x][i].w = e[i][x].w = 0;
  fill(b[x], b[x] + n + 1, 0);
  for (ll u : p[x]) {
    for (ll v = 1; v <= m; ++v)
      if (!e[x][v].w || d(e[u][v]) < d(e[x][v]))
        e[x][v] = e[u][v], e[v][x] = e[v][u];
    for (ll v = 1; v <= n; ++v)
      if (b[u][v]) b[x][v] = u;
  }
  ss(x);
}
void ex(ll u) {
  for (ll x : p[u]) ch(x, x);
  ll a = b[u][e[u][f[u]].u], r = gr(u, a);
  for (ll i = 0; i < r; i += 2) {
    ll x = p[u][i], y = p[u][i + 1];
    f[x] = e[y][x].u;
    s[x] = 1;
    s[y] = 0;
    sl[x] = 0;
    ss(y);
    ins(y);
  }
  s[a] = 1;
  f[a] = f[u];
  for (ll i = r + 1; i < p[u].size(); ++i)
    s[p[u][i]] = -1, ss(p[u][i]);
  st[u] = 0;
}
bool on(const Q &e) {
  ll u = st[e.u], v = st[e.v], a;
  if (s[v] == -1) {
    f[v] = e.u, s[v] = 1, a = st[lk[v]],
    sl[v] = sl[a] = s[a] = 0, ins(a);
  } else if (!s[v]) {
    a = lca(u, v);
    if (!a) return aug(u, v), aug(v, u), 1;
    else add(u, a, v);
  }
  return 0;
}
bool bfs() {
  fill(s, s + m + 1, -1);
  fill(sl, sl + m + 1, 0);  // s is filled with -1
  h = 1, t = 0;
  for (ll i = 1; i <= m; ++i)
    if (st[i] == i && !lk[i]) f[i] = s[i] = 0, ins(i);
  if (h > t) return 0;
  while (1) {
    while (h <= t) {
      ll u = q[h++];
      if (s[st[u]] != 1) {
        for (ll v = 1; v <= n; ++v)
          if (e[u][v].w > 0 && st[u] != st[v]) {
            if (d(e[u][v])) upd(u, st[v]);
            else if (on(e[u][v])) return 1;
          }
      }
    }
    ll x = inf;
    for (ll i = n + 1; i <= m; ++i)
      if (st[i] == i && s[i] == 1) x = min(x, lab[i] / 2);
    for (ll i = 1; i <= m; ++i)
      if (st[i] == i && sl[i] && s[i] != 1)
        x = min(x, d(e[sl[i]][i]) >> s[i] + 1);
    for (ll i = 1; i <= n; ++i)
      if (~s[st[i]])
        if ((lab[i] += (s[st[i]] * 2 - 1) * x) <= 0)
          return 0;
    for (ll i = n + 1; i <= m; ++i)
      if (st[i] == i && ~s[st[i]])
        lab[i] += (2 - 4 * s[st[i]]) * x;
    h = 1, t = 0;
    for (ll i = 1; i <= m; ++i)
      if (st[i] == i && sl[i] && st[sl[i]] != i &&
          !d(e[sl[i]][i]) && on(e[sl[i]][i]))
        return 1;
    for (ll i = n + 1; i <= m; ++i)
      if (st[i] == i && s[i] == 1 && !lab[i]) ex(i);
  }
}

pair<ll, vector<array<ll, 2>>> solve(
    ll N, vector<array<ll, 3>> edges) {
  for (auto &edge : edges) ++edge[0], ++edge[1];
  fill(ed, ed + m + 1, 0);
  fill(lk, lk + m + 1, 0);
  n = m = N;
  id = 0;
  iota(st + 1, st + n + 1, 1);
  ll wm = 0, weight = 0;
  for (ll i = 1; i <= n; ++i)
    for (ll j = 1; j <= n; ++j) e[i][j] = {i, j, 0};
  for (auto edge : edges) {
    int u = edge[0], v = edge[1];
    ll w = edge[2];
    wm = max(wm,
             e[v][u].w = e[u][v].w = max(e[u][v].w, w));
  }
  for (ll i = 1; i <= n; ++i) p[i].clear();
  for (ll i = 1; i <= n; ++i)
    for (ll j = 1; j <= n; ++j) b[i][j] = i == j ? i : 0;
  fill_n(lab + 1, n, wm);
  while (bfs()) {}
  vector<array<ll, 2>> matching;
  for (ll i = 1; i <= n; ++i)
    if (i < lk[i])
      weight += e[i][lk[i]].w,
          matching.push_back({i - 1, lk[i] - 1});
  return {weight, matching};
}
}  // namespace blossom
```

4.8 Floyd-Warshall Algorithm

```
int dis[N][N];
void init(int n) {
  for (int i = 0; i <= n; i++) {  // 初始化
    for (int j = 0; j <= n; j++)
      dis[i][j] = INF;
    dis[i][i] = 0;
  }
}
void floyd_warshall(int n) {
  for (int k = 0; k < n; k++) {  // 窮舉中繼點 k
    for (int i = 0; i < n; i++) {
      for (int j = 0; j < n; j++) {  // 窮舉點對 (i, j)
        dis[i][j] = min(dis[i][j], dis[i][k] + dis[k][j]);
      }
    }
  }
}
```

4.9 Maximum Clique

```
#define N 111       // 80 ~ 100 左右
struct MaxClique {  // 0-base
  typedef bitset<N> Int;
  Int linkto[N], v[N];
  int n;
  void init(int _n) {
    n = _n;
    for (int i = 0; i < n; i++) {
      linkto[i].reset();
      v[i].reset();
    }
  }
  void addEdge(int a, int b) { v[a][b] = v[b][a] = 1; }
  int popcount(const Int& val) { return val.count(); }
  int lowbit(const Int& val) { return val._Find_first(); }
  int ans, stk[N];
  int id[N], di[N], deg[N];
  Int cans;
  void maxclique(int elem_num, Int candi) {
    if (elem_num > ans) {
      ans = elem_num;
      cans.reset();
      for (int i = 0; i < elem_num; i++)
        cans[id[stk[i]]] = 1;
    }
    int potential = elem_num + popcount(candi);
    if (potential <= ans) return;
    int pivot = lowbit(candi);
    Int smaller_candi = candi & (~linkto[pivot]);
    while (smaller_candi.count() && potential > ans) {
      int next = lowbit(smaller_candi);
      candi[next] = !candi[next];
      smaller_candi[next] = !smaller_candi[next];
      potential--;
      if (next == pivot ||
          (smaller_candi & linkto[next]).count()) {
        stk[elem_num] = next;
        maxclique(elem_num + 1, candi & linkto[next]);
      }
    }
  }
  int solve() {
    for (int i = 0; i < n; i++) {
      id[i] = i;
      deg[i] = v[i].count();
    }
    sort(id, id + n, [&](int id1, int id2) {
      return deg[id1] > deg[id2];
    });
    for (int i = 0; i < n; i++)
      di[id[i]] = i;
    for (int i = 0; i < n; i++)
      for (int j = 0; j < n; j++)
        if (v[i][j]) linkto[di[i]][di[j]] = 1;
    Int cand;
    cand.reset();
    for (int i = 0; i < n; i++)
      cand[i] = 1;
    ans = 1;
    cans.reset();
    cans[0] = 1;
    maxclique(0, cand);
    return ans;
  }
} solver;
```

4.10 Kosaraju's Algorithm for SCC

```
struct Scc {  // 新的 DAG 可能會有多重邊
#define PB push_back
  int n, nScc, vst[MXN], bln[MXN];
  vector<int> E[MXN], rE[MXN], vec, dag[MXN];
  set<int> vis[MXN];
  void init(int _n) {
    n = _n;
    for (int i = 0; i <= n; i++) {
      E[i].clear(), rE[i].clear();
      dag[i].clear(), vis[i].clear();
      bln[i] = -1;
    }
  }
  void addEdge(int u, int v) {
    E[u].PB(v);
    rE[v].PB(u);
  }
  void DFS(int u) {
    vst[u] = 1;
    for (auto v : E[u])
      if (!vst[v]) DFS(v);
    vec.PB(u);
  }
  void rDFS(int u) {
    vst[u] = 1;
    bln[u] = nScc;  // 編號是 0-base
    for (auto v : rE[u])
      if (!vst[v]) rDFS(v);
  }
  void build_new_dag() {
    for (int i = 0; i < n; i++) {
      for (int &j : E[i]) {
        if (bln[i] != bln[j] &&
            vis[bln[i]].count(bln[j]) == 0) {
          dag[bln[i]].push_back(bln[j]);
          vis[bln[i]].insert(bln[j]);
        }
      }
    }
  }
  void solve() {
    nScc = 0;
    vec.clear();
    fill(vst, vst + n + 1, 0);
    for (int i = 0; i < n; i++)
      if (!vst[i]) DFS(i);
    reverse(vec.begin(), vec.end());
    fill(vst, vst + n + 1, 0);
    for (auto v : vec) {
      if (!vst[v]) {
        rDFS(v);
        nScc++;
      }
    }
    build_new_dag();
  }
};
```

4.11 Kruskal's Algorithm for MST

```
void kruskal() {
  sort(graph, graph + m);  // 將邊照大小排序
  int ans = 0;

  for (int i = 0; i < m; i++) {  //>:)
    if (find(graph[i].u) != find(graph[i].v)) {
      // 如果兩點未聯通
      merge(graph[i].u, graph[i].v);
      // 將兩點設成同一個集合
      ans += graph[i].w;  // 權重加進答案
      if (sz[find(graph[i].u)] == n)
        break;  // 當並查集大小等價於樹內點的數量
    }
  }
}
```

4.12 Kuhn-Munkre Algorithm

```
struct KM {  // max weight, for min negate the weights
  int n, mx[MXN], my[MXN], pa[MXN];
  ll g[MXN][MXN], lx[MXN], ly[MXN], sy[MXN];
  bool vx[MXN], vy[MXN];
  void init(int _n) {  // 1-based
    n = _n;            // 初始化
    for (int i = 1; i <= n; i++)
      fill(g[i], g[i] + n + 1, 0);
  }
  void addEdge(int x, int y, ll w) {  // 加邊
    g[x][y] = w;
  }
  void augment(int y) {
    for (int x, z; y; y = z)
      x = pa[y], z = mx[x], my[y] = x, mx[x] = y;
  }
  void bfs(int st) {
    for (int i = 1; i <= n; ++i)
      sy[i] = INF, vx[i] = vy[i] = 0;
```

```
    queue<int> q;
    q.push(st);
    for (;;) {
      while (q.size()) {
        int x = q.front();
        q.pop();
        vx[x] = 1;
        for (int y = 1; y <= n; ++y)
          if (!vy[y]) {
            ll t = lx[x] + ly[y] - g[x][y];
            if (t == 0) {
              pa[y] = x;
              if (!my[y]) {
                augment(y);
                return;
              }
              vy[y] = 1, q.push(my[y]);
            } else if (sy[y] > t) pa[y] = x, sy[y] = t;
          }
      }
      ll cut = INF;
      for (int y = 1; y <= n; ++y)
        if (!vy[y] && cut > sy[y]) cut = sy[y];
      for (int j = 1; j <= n; ++j) {
        if (vx[j]) lx[j] -= cut;
        if (vy[j]) ly[j] += cut;
        else sy[j] -= cut;
      }
      for (int y = 1; y <= n; ++y)
        if (!vy[y] && sy[y] == 0) {
          if (!my[y]) {
            augment(y);
            return;
          }
          vy[y] = 1, q.push(my[y]);
        }
    }
  }
  ll solve() {  // 解決
    fill(mx, mx + n + 1, 0);
    fill(my, my + n + 1, 0);
    fill(ly, ly + n + 1, 0);
    fill(lx, lx + n + 1, -INF);
    for (int x = 1; x <= n; ++x)
      for (int y = 1; y <= n; ++y)
        lx[x] = max(lx[x], g[x][y]);
    for (int x = 1; x <= n; ++x)
      bfs(x);
    ll ans = 0;
    for (int y = 1; y <= n; ++y)
      ans += g[my[y]][y];  // 答案存在 my[]
    return ans;
  }
} graph;  // 找二分圖最大權完美匹配
```

4.13 Max-flow-min-cost

```
struct zkwflow {
  static const int maxN = 100000;
  struct Edge {
    int v, f, re;
    ll w;
  };
  int n, s, t, ptr[maxN];
  bool vis[maxN];
  ll dis[maxN];
  vector<Edge> E[maxN];
  void init(int _n, int _s, int _t) {
    n = _n, s = _s, t = _t;
    for (int i = 0; i < n; i++)
      E[i].clear();
  }
  void add_edge(int u, int v, int f, ll w) {
    E[u].push_back({v, f, (int)E[v].size(), w});
    E[v].push_back({u, 0, (int)E[u].size() - 1, -w});
  }
  bool SPFA() {
    fill_n(dis, n, LLONG_MAX);
    fill_n(vis, n, false);
    queue<int> q;
    q.push(s);
    dis[s] = 0;
    while (!q.empty()) {
      int u = q.front();
      q.pop();
      vis[u] = false;
      for (auto &it : E[u]) {
        if (it.f > 0 && dis[it.v] > dis[u] + it.w) {
          dis[it.v] = dis[u] + it.w;
          if (!vis[it.v]) {
            vis[it.v] = true;
            q.push(it.v);
          }
        }
      }
    }
    return dis[t] != LLONG_MAX;
  }
  int DFS(int u, int nf) {
    if (u == t) return nf;
    int res = 0;
    vis[u] = true;
    for (int &i = ptr[u]; i < (int)E[u].size(); i++) {
      auto &it = E[u][i];
      if (it.f > 0 && dis[it.v] == dis[u] + it.w &&
          !vis[it.v]) {
        int tf = DFS(it.v, min(nf, it.f));
        res += tf, nf -= tf, it.f -= tf;
        E[it.v][it.re].f += tf;
        if (nf == 0) {
          vis[u] = false;
          break;
        }
      }
    }
    return res;
  }
  pair<int, ll> flow() {
    int flow = 0;
    ll cost = 0;
    while (SPFA()) {
      fill_n(ptr, n, 0);
      int f = DFS(s, INT_MAX);
      flow += f;
      cost += dis[t] * f;
    }
    return {flow, cost};
  }  // reset: do nothing
} flow;
```

4.14 Kuhn Algorithm for Maching

```
vector<int> g[MXN];
bool vis[MXN];
int S[MXN];  // S 為紀錄這個點與誰匹配
int n, ans;  // n: 左集合數量, ans: 紀錄答案

bool dfs(int u) {  // 找最大匹配
  for (int &v : g[u]) {
    if (!vis[v]) {
      vis[v] = true;
      if (S[v] == -1 || dfs(S[v])) {
        S[v] = u;
        return true;
      }
    }
  }
  return false;
}

void solve() {  // 記得每次使用需清空 vis
  memset(S, -1, sizeof(S));
  ans = 0;
  for (int i = 1; i <= n; i++) {  // 跑左集合
    memset(vis, 0, sizeof(vis));
    if (dfs(i)) ans++;
  }
}
```

5 Tree Algorithms

5.1 時間戳記 & 倍增表 (初始化)

```
const int LG_MXN = __lg(MXN) + 2;
vector<int> edge[MXN];  // i 有哪些小孩
int timing = 1, lgN, n;
int in[MXN], out[MXN], anc[MXN][LG_MXN], depth[MXN];
```

```
void dfs_init(int u, int f) {  // 0 倍祖先、時間戳記
  in[u] = timing++;            // 這時進入 u
  anc[u][0] = f;               // now 的 0 倍祖先是 f
  for (int v : edge[u]) {
    if (v == f) continue;
    depth[v] = depth[u] + 1;
    dfs_init(v, u);
  }
  out[u] = timing++;  // 這時離開 u
}
void build_table(int n) {  // 初始化
  lgN = __lg(n) + 1;
  dfs_init(1, 0);
  for (int i = 1; i <= lgN; i++)  // 建立倍增表
    for (int now = 1; now <= n; now++)
      anc[now][i] = anc[anc[now][i - 1]][i - 1];
}
```

5.2 LCA

```
bool is_ancestor(int u, int v) {  // 判斷 u 是否 v 是祖先
  return (in[u] < in[v] && out[v] < out[u]);
}
int getLCA(int x, int y) {  // 找最近共同祖先
  if (is_ancestor(x, y))    // 如果 x 為 y 的祖先
    return x;               // 則 LCA 為 y
  if (is_ancestor(y, x))    // 如果 y 為 x 的祖先
    return y;               // LCA 為 y
  for (int i = lgN; i >= 0; i--)  // 逼近 LCA
    if (!is_ancestor(anc[x][i], y) && anc[x][i])
      x = anc[x][i];
  return anc[x][0];  // 回傳此點的父節點即為答案
}
```

5.3 Eular Tour for LCA

```
#include <bits/stdc++.h>
using namespace std;
const int N = 1e5 + 5;
vector<int> tour, edge[N];
int timing = 0;
int in[N], out[N];
void dfs(int u, int fa) {
  tour.push_back(u);  // 進入的時間點
  in[u] = timing++;
  for (auto v : edge[u]) {
    if (v == fa) continue;
    dfs(v, u);
    tour.push_back(u);  // 走完其中一個孩子 再走回自己
  }
  out[u] = timing++;
}
void build(vector<int>& tour) {  // 把tour丟進線段樹初始化
  seg.build(tour);  // 除了tour，還要建每個點的深度
}
int query(int x, int y) {
  if (is_ancster(x, y)) return x;  // LCA(x,y) = x
  if (is_ancster(y, x)) return y;  // LCA(x,y) = y;
  // 如果「y的區間」在「x的區間」的左邊，就交換
  if (out[y] < in[x]) swap(x, y);
  // 取得區間內哪個位置的深度是最低的
  int index = seg.query(out[x], in[y]);
  return tour[index];
}
```

5.4 樹上差分

```
int tag[N], f[N], ans[N], root;
void add(int x, int y) {
  int lca = getLCA(x, y);
  tag[x]++;
  tag[y]++;
  tag[lca]--;
  if (lca != root) tag[f[lca]]--;
}
int dfs_add(int now, int fa) {
  int diff = tag[now];
  for (auto nxt : edge[now]) {
    if (nxt == fa) continue;
    diff += dfs_add(nxt, now);
  }
  return ans[now] = diff;
}
```

5.5 DSU on Tree

```
int find(int x) {
  if (f[x] == x) return x;
  else return f[x] = find(f[x]);
}
void merge(int x, int y) {
  x = find(x), y = find(y);
  if (sz[x] < sz[y]) swap(x, y);
  sz[x] += sz[y];
  f[y] = x;
}
void init() {
  // 初始化一開始大小都為1 (每群一開始都是獨立一個)
  for (int i = 1; i <= n; i++) sz[i] = 1;
  for (int i = 1; i <= n; i++) f[i] = i;
}
```

5.6 樹鏈剖分

```
#define REP(i, s, e) for (int i = (s); i <= (e); i++)
#define REPD(i, s, e) for (int i = (s); i >= (e); i--)
const int MAXN = 100010;
const int LOG = 19;
struct HLD {
  int n;
  vector<int> g[MAXN];
  int sz[MAXN], dep[MAXN];
  int ts, tid[MAXN], tdi[MAXN], tl[MAXN], tr[MAXN];
  //  ts : timestamp , useless after yutruli
  //  tid[ u ] : pos. of node u in the seq.
  //  tdi[ i ] : node at pos i of the seq.
  //  tl , tr[ u ] : subtree interval in the seq. of node
  //  u
  int prt[MAXN][LOG], head[MAXN];
  // head[ u ] : head of the chain contains u
  void dfssz(int u, int p) {
    dep[u] = dep[p] + 1;
    prt[u][0] = p;
    sz[u] = 1;
    head[u] = u;
    for (int& v : g[u])
      if (v != p) {
        dep[v] = dep[u] + 1;
        dfssz(v, u);
        sz[u] += sz[v];
      }
  }
  void dfshl(int u) {
    ts++;
    tid[u] = tl[u] = tr[u] = ts;
    tdi[tid[u]] = u;
    sort(ALL(g[u]),
         [&](int a, int b) { return sz[a] > sz[b]; });
    bool flag = 1;
    for (int& v : g[u])
      if (v != prt[u][0]) {
        if (flag) head[v] = head[u], flag = 0;
        dfshl(v);
        tr[u] = tr[v];
      }
  }
  inline int lca(int a, int b) {
    if (dep[a] > dep[b]) swap(a, b);
    int diff = dep[b] - dep[a];
    REPD(k, LOG - 1, 0) if (diff & (1 << k)) {
      b = prt[b][k];
    }
    if (a == b) return a;
    REPD(k, LOG - 1, 0) if (prt[a][k] != prt[b][k]) {
      a = prt[a][k];
      b = prt[b][k];
    }
    return prt[a][0];
  }
  void init(int _n) {
    n = _n;
    REP(i, 1, n) g[i].clear();
  }
  void addEdge(int u, int v) {
    g[u].push_back(v);
    g[v].push_back(u);
```